Unit – 6

VISIBLE SURFACE DETECTION

6.1 Hidden Surfaces and their Removal Techniques
6.2 Back-Face Detection
6.3 Depth Buffer Method
6.4 A-buffer Method
6.5 Scan Line Method
6.6 Area Subdivision Method
6.7 Depth Sorting Method

Visible surface detection

Visible surface detection, also known as visible surface determination or hidden surface removal, is a fundamental problem in computer graphics. It refers to the process of determining which surfaces or parts of surfaces in a three-dimensional (3D) scene are visible from a given viewpoint and should be displayed, while occluded or hidden surfaces are not rendered.

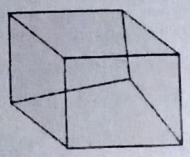
The goal of visible surface detection is to ensure that only the surfaces that contribute to the final image are rendered, improving the realism and efficiency of 3D graphics rendering.

- When we view a picture containing non-transparent objects and surfaces, then we cannot see those objects from view which are hidden from another the objects.
- These surfaces that are hidden or blocked from view must be removed to get a realistic view of 3D scene.
- The identification and removal of these surfaces is called **Hidden-surface problem**.
- The problem of hidden surface removal is solved by applying different algorithms.
- **These algorithms are broadly classified into two categories:**
  - Object space methods
  - Image space methods

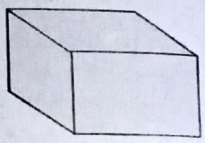
1. **Object Space Method**
   - Implemented in the world co-ordinate system, considering the geometrical relationship between the actual objects.
   - In this method, various parts of objects are compared.
   - After comparison visible, invisible or hardly visible surface is determined.

- So these algorithms are line based instead of surface based.
- Example:
  - Back-Face Detection
  - Depth Sorting (Painter Algorithm)
  - Roberts Algorithm.

2. **Image Space Method**
   - Implemented in the screen coordinate system, considering the relationship between the images of the objects in their final configurations at the pixel level.
   - It is used to locate the visible surface instead of a visible line.
   - Each point is detected for its visibility.
   - If a point is visible, then the pixel is on, otherwise off.
   - Example:
     - Depth Buffer (Z-buffer)
     - Scan-line
     - Area Subdivision



Object with hidden line



Object when hidden lines removed

Aspect	Object space method	Image space method
Definition	Operate directly on object geometry.	Operate on the rendered image or screen.
Visibility determination	Perform visibility tests on object geometry.	Compare pixel values or depth information.
Efficiency	Efficient for static or non-deforming objects.	Efficient for dynamic and deforming objects.
Complexity	Simpler algorithms and computations.	More complex algorithms and computations.
Occlusion Handling	Limited occlusion handling within objects.	Can handle inter-object and pixel-level occlusions.
Transparency support	Not well-suited for handling transparent objects.	Can handle transparency with appropriate techniques.

Aspect	Object space method	Image space method
Real-time performance	Efficient for scenes with a small number of objects.	Better suited for scenes with a large number of objects.
Implementation	Operate on the object's vertices, edges, and faces.	Operate on the pixel values or depth buffer of the rendered image.
Combination potential	Can be combined with other techniques for optimization.	Can be combined with other techniques for optimization.

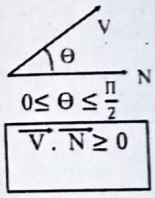
Back face Detection

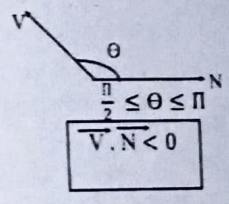
Back-face detection, also known as back-face culling, is a technique used in computer graphics to determine which faces of a 3D object are facing away from the viewer and should be culled or not rendered. The algorithm assumes that all faces of the object have consistent vertex order (either clockwise or counterclockwise) and that the object is closed.

- **Their relation can be represented by the dot product as below:**

$$\vec{V}.\vec{N} = |\vec{V}| . |\vec{N}| \cos\theta$$

Where θ is the angle between $\vec{V}$ and $\vec{N}$.





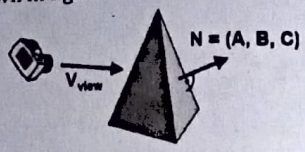
- Here, N and V are in same direction the face we detected is a backface.
- Here, N and V are in opposite direction the face we detected is a front face.

Back face detection & removal:

- Also known as Plane equation method.
- A fast and simple object-space method for identifying the back faces of a polyhedron is based on the "inside-outside" tests.
- A point (x, y, z) is "inside" a polygon surface with plane parameters A, B, C and D if

$A_x + B_y + C_z + D < 0$

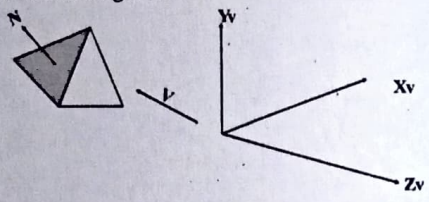
- When an inside point is along the line of sight to the surface, the polygon must be a back face that means the surface cannot be seen from the viewing position.
- We can simplify this test by considering the normal vector N to a polygon surface, which has Cartesian components A, B, C.
- And V is a vector in the viewing direction from the eye or camera position as shown in figure:



- Furthermore, if object descriptions are converted to projection coordinates and viewing direction is parallel to the viewing z-axis, then

$\vec{V} = (0,0,V_z)$ and $\vec{V}.\vec{N} = V_zC$

- In a right-handed viewing system with viewing direction along the Zv-axis as shown in figure:



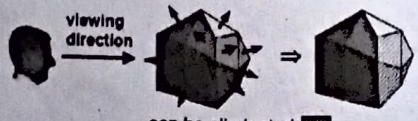
Here, $\vec{V}.\vec{N} = -Vz \cdot C$

For back face;

$V \cdot N \geq 0$

or, $-Vz \cdot C \geq 0$

- Here, the polygon is a back face if $C < 0$.
- Also, we cannot see any face whose normal has z component $C = 0$, since your viewing direction is towards that polygon.
- Thus, in general, we can label any polygon as a back face if its normal vector has a z component value: $C <= 0$



can be eliminated:

Advantages

- It is a simple and straight forward method.
- It reduces the size of databases, because no need of store all surfaces in the database, only the visible surface is stored.

Limitations

1. This method doesn't work well for overlapping faces, due to multiple objects or concave objects.
2. This method can only be used on solid objects modeled as a polygon mesh.

B. Depth sorting method

Makes use of both image and object space operations to:

i. Sort surfaces in decreasing depth order (using both image and object space).

ii. Scan converted in order starting with surface with greatest depth (using image space).

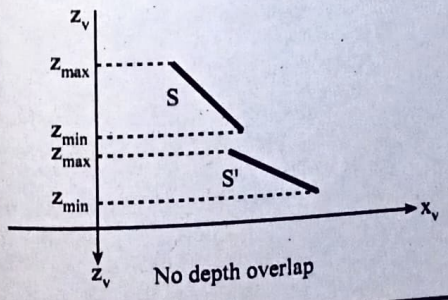
Also referred to as painters' algorithm because an artist first paints background color then most distant object then nearer object and so on. In the end the foreground objects are painted on canvas over the background. Each layer of paint covers up previous layer.

Similarly, we first sort surfaces according to their distance from view plane.

Intensity values for farthest surface are entered into refresh buffer.

Taking each surface in turn (in decreasing depth order) we paint surface intensities onto frame buffer over intensity of previously processed surface.

We assume we're viewing along 'z' direction. Surfaces are ordered according to largest 'z' value on each surface.

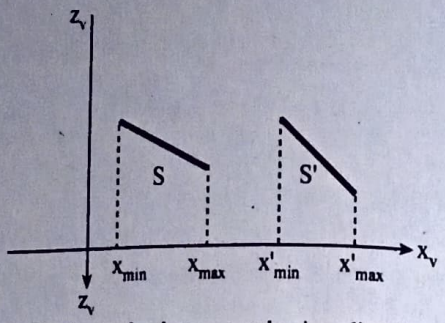


No depth overlap

Surface with greatest depth is compared with other surface in list to see if there are any overlaps in depth.

If no depth overlap then surfaces S is scan converted.

Additional test are required for each surface in case of depth overlap with surface S:



Depth overlap but no overlap in x direction

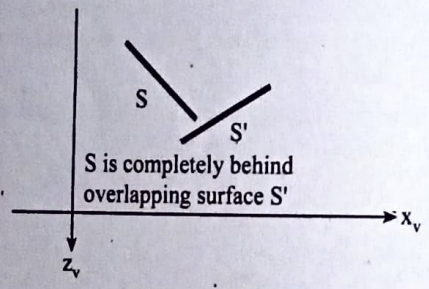
i. Bounding rectangle in xy plane for two surfaces don't overlap.

ii. Surface S is completely behind overlapping surface relative to viewing position.

iii. Overlapping surface S' is completely in-front of S relative to viewing position.

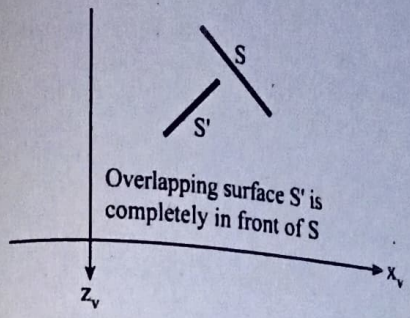
iv. Projection of 2 surfaces onto view plane don't overlap.

If any of these conditions is true then no reordering of surfaces is necessary. i.e. if all surfaces pass at least one of the tests then none of those surfaces are behind S.



For test (i) we perform two steps to check overlap first in 'x' then in 'y' direction. If either of these directions show no overlap then the two planes cannot obscure one another.

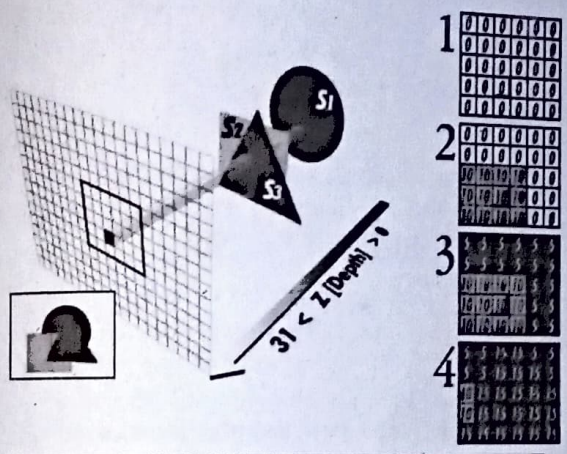
For tests (ii) and (iii), we can perform inside outside polygon test. Substitute coordinates for all vertices into plane equation for overlapping surfaces and check the sign of the result obtained.



Based on plane equation if all vertices of S are inside S' then S is completely behind S'. Similarly, S' is completely in-front of S if all vertices of S are outside of S'.

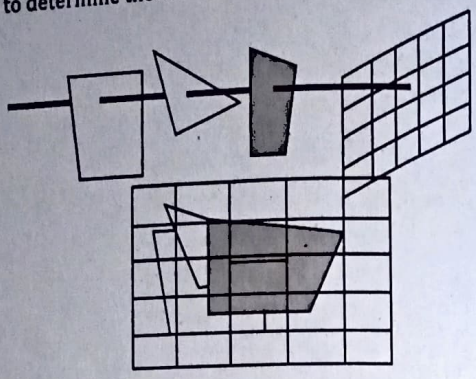
For test (iv) we can use line equation in xy plane for checking intersections between bounding edges of two surfaces.

Depth Buffer(Z–Buffer method)



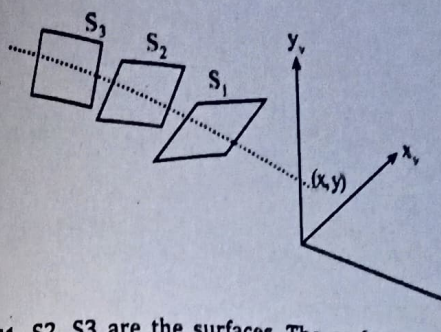
The Z-buffer, also known as the depth buffer, is a key component of visible surface determination in computer graphics. It is a 2D array that stores the depth or distance information for each pixel in the image or screen space.

- This method is developed by Catmull.
- Also known as the **Depth-buffer method.**
- An **Image space method**, commonly used method for hidden surface detection.
- Compares surface depths at each pixel position on the projection plane to determine the closest visible surface.



- Normally z-axis is represented as the depth.
- **Uses TWO buffer:**
  1. Z-buffer
  2. Frame buffer

1. **Z-Buffer:**
   - Used to store the z-coordinate of each calculated pixel values of a surface.
   - Also called as **depth buffer.**
2. **Frame buffer:**
   - Used to store the intensity values of each calculated pixel values.
   - Also called as **refresh buffer.**
   - We can implement the depth buffer algorithm in normalized co-ordinate so that the Z-values range from 0 to Z_{max}.
   - The value of Z_{max} can be set to either one (for unit cube) or the largest value that can be stored in the system.



- Let S1, S2, S3 are the surfaces. The surface closest to projection plane is called visible surface.
- The algorithm would start with surface 1 and put its value into the buffer.
- It does the same for the next surface.
- It checks each overlapping pixel and check to see which one is closer to the viewer and then display the appropriate color.
- As at view-plane position (x, y), surface S1 has the smallest depth from the view plane, so it is visible at that position.

Algorithm

- Initialize frame buffer to background colour
- Initialize z buffer to minimum z value.
- Scan convert each polygon in arbitrary order.
- For each (x,y) pixel calculate depth 'z' at that pixel z(x,y)
- Compare calculated new depth z(x,y) with value previously stored in z- buffer at that location z(x,y)
- If z(x,y) > z(x,y) then write the new depth value to z-buffer and update frame buffer.
- Otherwise no action is taken.
- **The algorithm can be summarized as below:**

Step 1: Initialize the depth value and intensity value for each pixel point (x , y) on a surface as below:

Zbuffer (x , y) = maximum value (Say ∞)

$Frame_{buffer}$ (x , y) = background color($I_{background}$)

Step 2: Process each polygon One at a time.

- For each projected (x , y) pixel position of a polygon, calculate depth say $Z_{calc}(x,y)$.
- If $Z_{calc}(x,y) < Z_{buffer}(x,y)$

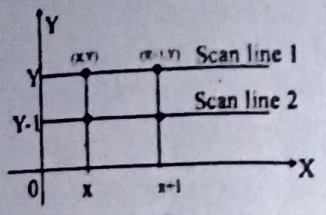
then set $Z_{buffer}(x,y) = Z_{calc}(x,y)$

and $Refresh_{buffer} = I_{calc}(x,y)$

Step3: After all the pixel over the polygon surfaces are compared, draw the object using (x ,y) from the depth and intensity buffer.

Depth calculation:

- The depth value for any point on the surface of polygon whose plane equation is $A_x + B_y + C_z + D = 0$ is given by;
- $Z = \frac{-(A_x + B_y + D)}{C}$



Then, for next pixel value (X+1,Y)

$$Z' = \frac{-[A(X+1) + BY + D]}{C}$$

$$= \frac{-(AX + A + BY + D)}{C}$$

$$= \frac{-(AX + BY + D)}{C} - \frac{A}{C}$$

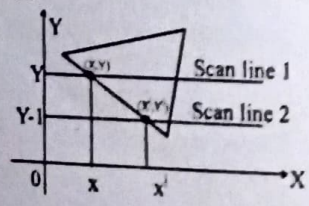
$$\therefore Z' = Z - \frac{A}{C}$$

The computation can be reduced to a single add per pixel using an incremental method. That means the depth value of the next position (x+1,y) on the scan line can be obtained using

Where A/C is a constant.

So succeeding depth values across a scan line are obtained from preceding values with a single addition.

- Similarly, starting at the top-vertex, we can recursively calculate the left edge of a polygon as:



- Here, $m = \frac{\Delta Y}{\Delta X} = \frac{((Y-1)-Y)}{X'-X} = \frac{-1}{X'-X}$

or, $X' - X = -\frac{1}{m}$

$\therefore X' = X - \frac{1}{m}$

Then at pixel value (X', Y-1), the value of Z can be calculated as

$$Z' = \frac{-[AX' + B(Y-1) + D]}{C}$$

$$= \frac{-[A(X - \frac{1}{m}) + BY - B + D]}{C}$$

$$= \frac{-(AX - \frac{A}{m} + BY - B + D)}{C}$$

$$= \frac{-(AX + BY + D)}{C} + \frac{\frac{A}{m} + B}{C}$$

$$\therefore Z' = Z + \frac{\frac{A}{m} + B}{C}$$

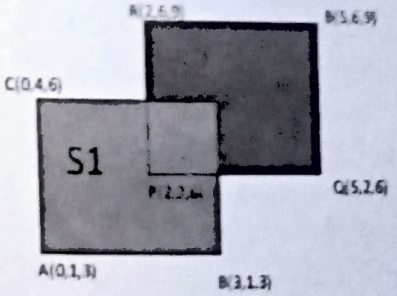
We are processing down a vertical edge;

$$Z' = Z + \frac{\frac{A}{m} + B}{C} \quad [\because m \to \infty \text{ since } \theta \to 90]$$

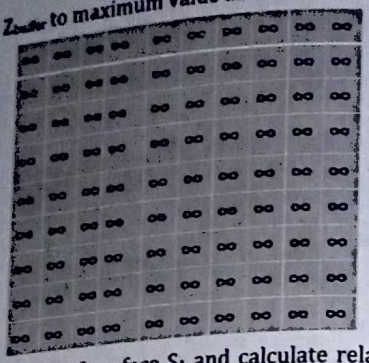
$$\therefore Z' = Z + \frac{B}{C}$$

Hence, succeeding depth values across a scan line are obtained from preceding values with a single addition.

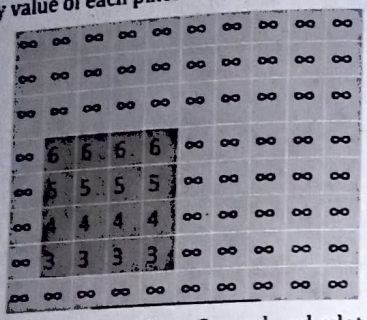
- Let us consider two polygon surfaces (non-transparent) S_1 and S_2 as shown in figure.



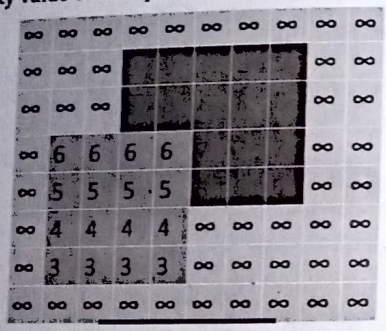
Step 1: Set Z_{buffer} to maximum value and $refresh_{buffer} = I_{background}$.



- Scan each pixel of surface S_1 and calculate relative z-value and intensity value of each pixel of that surface.



- Scan each pixel of surface S_2 and calculate relative z and intensity value of each pixel of that surface.

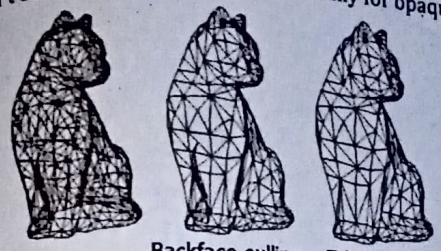


Advantages

- No presorting and object-object comparison.
- It is simple and easy to implement.
- Good for animation.

Limitations:

- Large memory to process each pixels.
- Time consuming.
- Doesn't suit for transparent object i.e. only for opaque one.
-



Backface culling Z-buffer algorithm

A buffer method

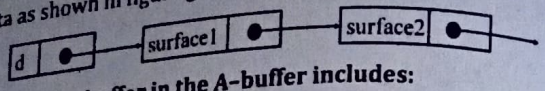
In computer graphics, a buffer method refers to a technique that utilizes buffers to store and process different types of information during the rendering pipeline. Buffers are temporary storage areas in computer memory used to hold data during graphics operations.

- A-buffer method is an extension of the z-buffer method.
- Represents an anti-aliased, area average, accumulation buffer method developed at Lucas film Studios for the rendering system called **REYES** (Renders Everything You Ever Saw).
- The A-buffer expands on the depth buffer method to allow transparencies.
- The key data structure in the A-buffer is the **Accumulation** buffer.
- Each position in the buffer can reference a linked list of surfaces.
- More than one surface intensity can be taken into consideration at each pixel position.
- **Each position in the A-buffer has two fields:**
  1. **Depth field:** It stores a positive or negative real number values.
  2. **Intensity field:** It stores surface-intensity information or a pointer value.
- **Example:**

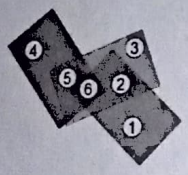
Depth field value (d)	Intensity field value (I)

  - If d >= 0, the number stored at that position is the depth of a single surface overlapping the corresponding pixel area.
  - The intensity field then stores the RGB components of the surface color at that point and the percent of pixel coverage.

- If depth < 0, it indicates multiple-surface contributions to pixel intensity.
- The intensity field then stores a pointer to a linked list of surface data as shown in figure given below.



- **The surface buffer in the A-buffer includes:**
  - RGB intensity components
  - Opacity Parameter
  - Depth
  - Percent of area coverage
  - Surface identifier
  - Other surface rendering parameter.
- The algorithm proceeds just like the depth buffer algorithm. The depth and opacity values are used to determine the final color of a pixel.



① Red
② Red
③ Green
④ Blue
⑤ Green
⑥ Red

1. R
2. G → R
3. G
4. B
5. G → B
6. G → B → R

Aspect	Z-Buffer method	A-Buffer method
Buffer type	Depth buffer that stores per-pixel depth values	Accumulation buffer that stores per-pixel fragment details
Visibility Determination	Compares depth values to determine visibility	Stores and accumulates all fragments for each pixel
Memory usage	Requires memory for depth buffer only	Requires memory for multiple buffers for each pixel
Occlusion handling	Handles occlusion naturally	Requires additional processing to handle occlusion

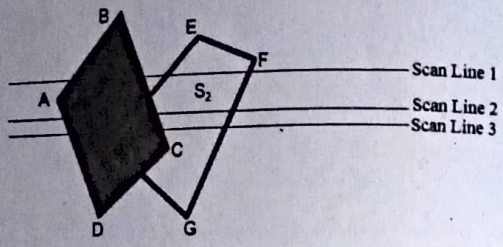
Aspect	Z-Buffer method	A-Buffer method
Transparency support	Limited support for transparent objects	Can handle transparency with appropriate techniques
Anti-aliasing	Limited anti-aliasing support	Can achieve higher-quality anti-aliasing effects
Fragment complexity	Performs well for scenes with simple geometry	More memory and processing-intensive for complex scenes
Order dependence	Independent of object order	Requires handling of fragment ordering and blending
Performance	Efficient for real-time rendering	Can be more computationally expensive for large scenes
Implementation	Relatively simple implementation	Requires additional buffer management and processing

Scanline method

The scanline method, also known as scanline rendering or scanline algorithm, is a technique used in computer graphics for rendering 2D or 3D scenes. It involves traversing the image or screen space line by line, or scanline by scanline, and determining the visible surfaces or polygons at each scanline.

- An image-space method to identify visible surface.
- This method has a depth information for only single scan-line.
- In order to require one scan-line of depth values, we must group and process all polygons intersecting a given scan-line at the same time before processing the next scan-line.
- Two important tables, **edge table** and **polygon table,** are maintained for this.
  - **The Edge Table** – It contains coordinate endpoints of each line in the scene, list of active edges(which are crossed by the scan lines), the inverse slope of each line, and pointers into the polygon table to connect edges to surfaces.
  - **The Polygon Table** – It contains the plane coefficients, intensity information about the surface, surface material properties, other surface data, and may be pointers to the edge table.

- Also a flag is set either ON or OFF for each surface to indicate whether a position along a scan-line is either inside or outside the surface.
- Pixel positions across each scan-line are processed from left to right.
- At the left intersection with a surface, the surface flag is turned on and at the right, the flag is turned off. You only need to perform depth calculations when multiple surfaces have their flags turned on at a certain scan-line position.
- Figure shows two surfaces S1 and S2. Surface S1 is above and surface S2 is below the surface S1. Some part of S2 is overlapped as shown in figure. Here s1 is nearer than S2 from view plane.
- **For scanline1:**



  - Here, active edges are AB, BC, EH and FG, stored in edge table.
  - For positions along this scan line between edges AB and BC, only the flag for surface S1 is ON. Therefore no depth calculation is necessary and intensity information for S1 is entered from the polygon table into the refresh buffer.
  - Similarly, between edges EH and FG, only the flag for surface S2 is ON. Intensity information for S2 is entered from the polygon table into the refresh buffer.
- **For scanline2**
  - Here, the active edges are AD, EH, BC, and FG, stored in edge table.
  - Along scan line 2 from edge AD to edge EH, only the flag for surface S1 is ON. But between the edges EH and BC, the flags for both surfaces are ON. In this interval. Depth calculations must be made using the plane coefficients for the two surfaces as following:

$$Z_1 = \frac{-(AX_1 + BY_1 + D)}{C} \text{ and } Z_2 = \frac{-(AX_2 + BY_2 + D)}{C}$$

- Now, the depths of both surfaces are compared.

- For this example, the depth of surface S1 is assumed to be less than that of S2 i.e. z1<z2, so intensities for surface s1 are loaded into the refresh buffer until the boundary BC is encountered.
- Then the flag for surface S1 goes OFF and intensity for surface S2 is stored into the refresh buffer until edge FG is passed.

Area-subdivision methods

This technique for hidden-surface removal is essentially an image-space method, but object-space operations can be used to accomplish depth ordering of surfaces.

The area-subdivision method takes advantage of area coherence (consistency) in a scene by locating those view areas that represent part of a single surface.

This method is applied successively by dividing the total viewing area into smaller and smaller rectangles until each small area is the projection of part of n single visible surface or no surface at all.

To implement this method, tests need to establish that can quickly identify the area as part of a single surface or show that the area is too complex to analyze easily.

Starting with the total view, apply the tests to determine whether subdivision of the total area must be done into smaller rectangles.

If the tests indicate that the view is sufficiently complex, subdivide it. Next, apply the tests to each of the smaller areas, subdividing these if the tests indicate that visibility of a single surface is still uncertain.

Continue this process until the subdivisions are easily analyzed as belonging to a single surface or until they are reduced to the size of a single pixel.

An easy way to do this is to successively divide the area into four equal parts at each step.

This approach is similar to that used in constructing a quadtree. A viewing area with a resolution of 1024 by 1024 could be subdivided ten times in this way before a subarea is reduced to a pint.

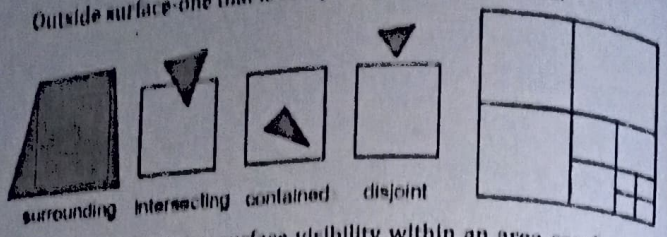
Tests to determine the visibility of a single surface within a specified area are made by comparing surfaces to the boundary of the area.

There are four possible relationships that a surface can have with a specified area boundary.

Relative surface characteristics:

- Surrounding surface-one that completely encloses the area.
- Overlapping surface-one that is partly inside and partly outside the area.

- Inside surface-one that is completely inside the area.
- Outside surface-one that is completely outside the area.



The tests for determining surface visibility within an area can be stated in terms of these four classifications. No further subdivisions of a specified area are needed if one of the following conditions is true:

1. All surfaces are outside surfaces with respect to the area.
2. Only one inside, overlapping, or surrounding surface is in the area.
3. A surrounding surface obscures all other surfaces within the area boundaries.

Test 1 can be carried out by checking the boundary rectangles of all surfaces against the area boundaries.

Test 2 can also use the bounding rectangles in the xy plane to identify an inside surface.

For other types of surfaces, the bounding rectangles can be used as an initial check. If a single bounding rectangle intersects the area in some way, additional checks are used to determine whether the surface is surrounding, overlapping, or outside.

Once a single inside, overlapping, or surrounding surface has been identified, its pixel intensities are transferred to the appropriate area within the frame buffer.

One method for implementing test 3 is the order surfaces according to their minimum depth from the view plane. For each surrounding surface, we then compute the maximum depth within the area under consideration. If the maximum depth of one of these surrounding surfaces is closer to the view plane than the minimum depth of all other surfaces within the area, test 3 is satisfied.

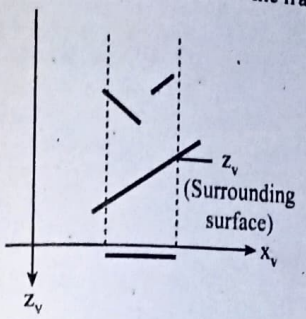
Another method for carrying out test 3 that does not require depth sorting is to use plane equations to calculate depth values at the four vertices of the area for all surrounding, overlapping, and inside surfaces, if the calculated depths for one of the surrounding surfaces is less than the calculated depths for all other surfaces, test 3 is true. Then the area can be filled with the intensity values of the surrounding surface.

For some situations, both methods of implementing test 3 will fail to identify correctly a surrounding surface that obscures all the other surfaces. Further testing could be carried out to identify the single surface that covers the area, but it is faster to subdivide the area than to continue with more complex testing.

Once outside and surrounding surfaces have been identified for an area, they will remain outside and surrounding surfaces for all subdivisions of the area.

Furthermore, some inside and overlapping surfaces can be expected to be eliminated as the subdivision process continues, so that the areas become easier to analyze.

In the limiting case, when a subdivision the size of a pixel is produced, we simply calculate the depth of each relevant surface at that point and transfer the intensity of the nearest surface to the frame buffer.



Algorithm:

- Try to make an easy decision about which polygon is visible in a section of the image. If a decision cannot be made, subdivide the area recursively until one can be made.
- Work at image precision for subdivision, and at object precision for depth comparison

For each area do the following tests:

1. are all polygons disjoint from area?

   if yes, display background color
2. only one intersecting or contained polygon.

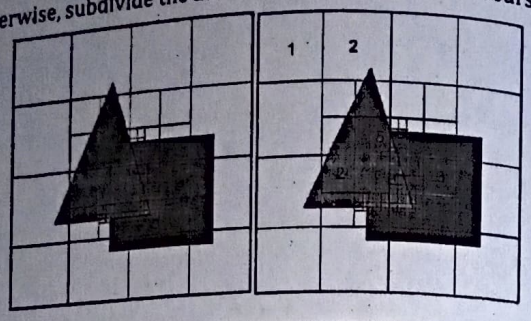
   if yes, fill with background color, and then draw contained polygon or intersecting portion
3. one single surrounding polygon, no intersecting or contained polygons.

   if yes, draw area with that polygon's color

4. more than one polygonis intersecting, contained in, or surrounding, but only one polygon is surrounding the area and is in front of others.

   If yes, draw area with that front polygon's color

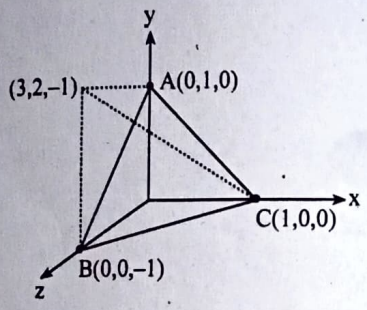
5. otherwise, subdivide the area into 4 equal areas and recurse



Solved Numerical Problems

1. **A plane is defined with the vertices A(0,1,0), B(0,0,–1) and C(1,0,0), if the observer is at location P(3,2,–1) will the surface be a back face to the observer from the given location?**

Solution:



$\vec{AB} = \vec{OB} - \vec{OA}$

$= (0,0,-1) - (0,1,0)$

$= (0-0)\vec{i} + (0-1)\vec{j} + (-1-0)\vec{k}$

$= -\vec{j} - \vec{k}\ (0,-1,-1)$

$\vec{AC} = \vec{OC} - \vec{OA}$

$= (1,0,0) - (0,1,0)$

$= (1-0)\vec{i} + (0-1)\vec{j} + (0-0)\vec{k}$

$= \vec{i} - \vec{j}\ (1,-1,0)$

$n = \vec{AB} \times \vec{AC}$

$= \begin{bmatrix} \vec{i} & \vec{j} & \vec{k} \\ 0 & -1 & -1 \\ 1 & -1 & 0 \end{bmatrix}$

$= ((0-(-1)\times(-1))\vec{i} + (0-1\times(-1))\vec{j} + (0-(1)\times(-1))\vec{k}$

$= -\vec{i} + \vec{j} + \vec{k}$

Choosing a point outside pyramid P(3,2,–1).

The AP vector:

$\vec{AP} = (3-0)\vec{i} + (2-1)\vec{j} + (-1-0)\vec{k}$

$= 3\vec{i} + \vec{j} + \vec{k}$

$\therefore\ n.\vec{AP} = (-\vec{i} + \vec{j} + \vec{k}).(3\vec{i} + \vec{j} - \vec{k})$

$= -3 + 1 - 1$

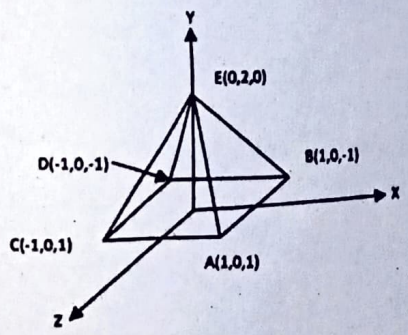
$= -3 < 0$

Hence, the V.N < 0

The normal is pointing outward, hence is a backface or invisible.

2. **Find the visibility for the surface BED, ABCD, ABE, and ACE where observer at P(5,5,5).**

Solution:



For surface BED,

Normal vector of the surface BED, $\vec{N_{BED}} = \vec{BD} \times \vec{BE}$

$= \{(-1-1)\vec{i} + (0-0)\vec{j} + (-1+1)\vec{k}\} \times \{(0-1)\vec{i} + (2-0)\vec{j} + (0-1)\vec{k}\}$

$= (-2\vec{i}) \times (-\vec{i} + 2\vec{j} + \vec{k})$

$= 2\vec{j} - 4\vec{k}$

Now,

$\vec{V_{PE}} = -5\vec{i} - 3\vec{j} - 5\vec{k}$

$\vec{V_{PB}} = -4\vec{i} - 5\vec{j} - 6\vec{k}$

$\vec{V_{PD}} = -6\vec{i} - 5\vec{j} - 6\vec{k}$

Now, for visibility test for vertex E,

$\vec{V_{PE}} \times \vec{N_{BED}} = (-5\vec{i} - 3\vec{j} - 5\vec{k}) \times (2\vec{j} - 4\vec{k})$

$= -6 + 20$

$= 14 > 0$

So, the vertex E is invisible

For Visibility test for vertex B,

$\vec{V_{PB}} \times \vec{N_{BED}} = (-4\vec{i} - 5\vec{j} - 6\vec{k}) \times (2\vec{j} - 4\vec{k})$

$= -10 + 24$

$= 14 > 0$

So, the vertex B is invisible

For Visibility test for vertex D,

$\vec{V_{PD}} \times \vec{N_{BED}} = (-6\vec{i} - 5\vec{j} - 6\vec{k}) \times (2\vec{j} - 4\vec{k})$

$= -10 + 24$

$= 14 > 0$

So, the vertex D is invisible

For surface ABCD,

Normal vector of the surface ABCD,

$\vec{N_{ABCD}} = \vec{BA} \times \vec{BD}$

$= (2\vec{k}) \times (-2\vec{i})$

$= -4\vec{j}$

Now,

$\vec{V_{PB}} = -4\vec{i} - 5\vec{j} - 6\vec{k}$

$\vec{V_{PD}} = -6\vec{i} - 5\vec{j} - 6\vec{k}$

$\vec{V_{PA}} = -4\vec{i} - 5\vec{j} - 4\vec{k}$

$\vec{V_{PC}} = -6\vec{i} - 5\vec{j} - 4\vec{k}$

For visibility test for vertex B,

$\vec{V_{PB}} \times \vec{N_{ABCD}} = (-4\vec{i} - 5\vec{j} - 6\vec{k}) \times (-4\vec{j})$

$= 20$

$= 20 > 0$

So, the vertex B is invisible

For visibility test for vertex D,

$\vec{V_{PD}} \times \vec{N_{ABCD}} = (-6\vec{i} - 5\vec{j} - 6\vec{k}) \times (-4\vec{j})$

$= 20$

$= 20 > 0$

So, the vertex D is invisible

Now, For visibility test for vertex A,

$\vec{V_{PA}} \times \vec{N_{ABCD}} = (-4\vec{i} - 5\vec{j} - 4\vec{k}) \times (-4\vec{j})$

$= 20$

$= 20 > 0$

So, the vertex A is invisible

Now, For visibility test for vertex ,

$\vec{V_{PC}} \times \vec{N_{ABCD}} = (-6\vec{i} - 5\vec{j} - 4\vec{k}) \times (-4\vec{j})$

$= 20$

$= 20 > 0$

So, the vertex C is invisible

For surface ABE,

Normal vector of the surface ABE,

$\vec{N_{ABE}} = \vec{AE} \times \vec{AB}$

$= (-\vec{i} + 2\vec{j} - \vec{k}) \times (2\vec{j})$

$= 4\vec{i} + 2\vec{j}$

Now,

$\vec{V_{PE}} = -5\vec{i} - 3\vec{j} - 5\vec{k}$

$\vec{V_{PA}} = -4\vec{i} - 5\vec{j} - 4\vec{k}$

$\vec{V_{PB}} = -4\vec{i} - 5\vec{j} - 6\vec{k}$

For visibility test for vertex E,

$\vec{V_{PE}} \times \vec{N_{ABE}} = (-5\vec{i} - 3\vec{j} - 5\vec{k}) \times (4\vec{i} + 2\vec{j})$

$= -20 - 6$

$= -26 < 0$

So, the vertex E is visible

For visibility test for vertex B,

$\vec{V_{PB}} \times \vec{N_{ABE}} = (-4\vec{i} - 5\vec{j} - 6\vec{k}) \times (4\vec{i} + 2\vec{j})$

$= -16 - 20$

$= -36 < 0$

So, the vertex B is visible

Now, for visibility test for vertex A,

$\vec{V_{PA}} \times \vec{N_{ABE}} = (-4\vec{i} - 5\vec{j} - 4\vec{k}) \times (4\vec{i} + 2\vec{j})$

$= -16 - 20$

$= -36 < 0$

So, the vertex A is visible

For surface ACE,

Normal vector of the surface ACE,

$\vec{N_{ACE}} = \vec{AE} \times \vec{AC}$

$= (\vec{i} + 2\vec{j} - \vec{k}) \times (-2\vec{j})$

$= 3\vec{i} + 4\vec{j}$

Now,

$\vec{V_{PA}} = -4\vec{i} - 5\vec{j} - 4\vec{k}$

$\vec{V_{PC}} = -6\vec{i} - 5\vec{j} - 4\vec{k}$

$\vec{V_{PE}} = -5\vec{i} - 3\vec{j} - 5\vec{k}$

For visibility test for vertex A,

$\vec{V_{PA}} \times \vec{N_{ACE}} = (-4\vec{i} - 5\vec{j} - 4\vec{k}) \times (3\vec{i} + 4\vec{j})$

$= -15 - 16$

$= -31 < 0$

So, the vertex A is visible

For visibility test for vertex C,

$\vec{V_{PC}} \times \vec{N_{ACE}} = (-6\vec{i} - 5\vec{j} - 4\vec{k}) \times (3\vec{i} + 4\vec{j})$

$= -15 - 16$

$= -31 < 0$

So, the vertex C is visible

For visibility test for vertex E,

$\vec{V_{PE}} \times \vec{N_{ACE}} = (-5\vec{i} - 3\vec{j} - 5\vec{k}) \times (3\vec{i} + 4\vec{j})$

$= -9 - 20$

$= -29 < 0$

So, the vertex E is visible.